

SAS/R Monitoring & On-Leave Coverage

*Deterministic, scheduled SAS/R that keeps the trial monitored and the tracker current — so nothing slips while you're out on leave. **No AI in the loop:** it detects, packages, and pages; humans decide.*

The one rule. The automation surfaces *pre-specified deterministic flags* and routes them to people. It does **not** triage, interpret, adjudicate, or auto-act on safety. The medical monitor and the covering biostatistician make every call; reported numbers come from validated tools (Phoenix WinNonlin, Pinnacle 21, the validated pipeline) with independent double-programming; the optional language model never produces a number. No PHI / unblinded data in the tracker.

The dedicated wiki

SAS/R Monitoring & On-Leave Coverage — the full wiki

One navigable wiki at the same depth as the operating wiki: the daily-job architecture (ingest → freshness gate → checks → roll-up → digest → tier-2 alert → heartbeat → watchdog), the deterministic safety checks (eDISH/Hy's-Law, QTcF, AE/SAE, DLT-vs-3+3, labs), alerting & the escalation matrix, the **covering-while-on-leave runbook**, the Smartsheet tracker, the optional on-device language layer, the macro libraries, safeguards, and IT readiness.

[Open the wiki \(HTML\)](#)[PDF](#)

Covering while out on leave — the headline. Everything runs under a service account (not your laptop); a named backup is paged on any RED with a one-page evidence packet; the tracker stays current; and an independent dead-man's switch makes “no email” itself a signal. Hand your cover one page and go.

[The on-leave runbook \(wiki \\$\)](#)[Coverage Runbook \(PDF\)](#)[▶ Watch the live screencast](#)[▶ Annotated walkthrough](#)

See it work — the interactive dashboard

A self-contained, click-through dashboard on a **synthetic** CP-101 run — the deterministic backbone made visible (the eDISH / Hy's-Law screen, QTcF tiers, AE/SAE, DLT-vs-3+3, the freshness gate) with a click-to-open participant evidence packet.

TRIALMON — Interactive Monitoring Dashboard

The **deterministic backbone**, made visible (validated logic): the signature plots and the escalation matrix on a seeded synthetic cohort with planted teaching signals — a Hy's-Law RED, a QTcF RED, a DLT cluster vs 3+3, and an instructive near-miss the AND-gate correctly keeps GREEN. Click any participant for the in-browser analog of the evidence packet. Runs offline in any browser; no data leaves the file.

[Open the dashboard](#)[Field Guide](#)[What it shows \(wiki §\)](#)

Honest by design. The dashboard runs on **synthetic** data (no PHI, no real participants); its flags are *screening prompts*, never determinations or reportable numbers; reported numbers come from validated tools (Phoenix WinNonlin / Pinnacle 21). It ships with a dedicated, operating-depth [Field Guide](#) documenting every section, panel, threshold and governance rule.

The three components, in depth

TRIALMON · the monitoring backbone

The deterministic SAS/R loop — freshness gate, the safety + operational checks, the tier-2 alert with an evidence packet, the heartbeat, and the independent watchdog. The `%tm_*` library + R companion.

[Open wiki](#)[Written in SAS \(screencast\)](#)[Macro README](#)

SHEETLINK · the tracker stays current

The same scheduled job upserts status into the Smartsheet program tracker — idempotent by key, ops-only allowlist, token from a secret. Independently SAS-reviewed. The `%ss_*` library.

[Open wiki](#)[Macro README](#)[Live screencast](#)

LOCALMIND · optional language layer

An *optional* on-prem SLM for the language tasks code can't do, behind loopback + validator + human-gate guardrails. The monitoring needs none of it. The `%slm_*` library + the IT enablement runbook.

[Open wiki](#)[IT runbook](#)[Example](#)

Deterministic, no-AI monitoring — the lowest-risk, ship-today pipeline in the program: it detects, packages, and pages; humans decide. Validate the threshold logic + scheduling wrapper as study programs; double-program anything reported. The optional on-device SLM is ops-only and behind guardrails.